an independent storage engine, whereas most other time series databases depend on other storage back-ends. Recent versions of InfluxDb are built on a Time Structured Merge (TSM) based BoltDB engine, which is a shift from LSM based LevelDB from v0.9.x.

The TICK stack comes with the following four components, with its unique ecosystem for complete IoT data management and integration with other solutions:

Telegraf	Data collection
InfluxDb	Data storage
Chronograf	Data visualisation
Kapacitor	Data processing, alerting and ETL jobs

This article is based on v1.2.0 of the above components, which is the latest at present (even though Telegraf v1.2.1 is available). All these components are available in rpm, deb package formats for 64-bit Linux platforms and binaries for 64-bit Windows, 32-bit Linux and ARM platforms are also available. Official Docker images are also available for the above components, These images can be started and linked together using Docker Compose using the hints given at *https://github.com/influxdata/TICK-docker*. One can build a custom Docker image using Docker build scripts available at *https://github.com/influxdata/influxdata-docker* or with the help of available rpm, deb packages using a base OS like Debian, Ubuntu or CentOS. You can also build these from sources using the GO build system for desired versions, which generates single binaries without external dependencies and templates for configuration files and necessary scripts.

Let's now have a walkthrough of InfluxDb.

InfluxDb

Once InfluxDb is installed, you can launch the server using the *influxd* binary directly or using *init* control systems like *systemctl* based on *init.d* or *systemd* scripts, with the default configuration file being available at */etc/influxdb/infludb.conf*. To enable the Web admin interface, modify the configuration file as follows and restart the server daemon:

```
[admin]
enabled=true
bind-address = ":8083"
```

Note: Please refer to Romin Irani's article on InfluxDb published in the November 2016 edition of *OSFY* for initial pointers like database creation, and writing data and basic queries using the Influx CLI tool, Web console or HTTP REST APIs.

Additionally, InfluxData comes with various libraries to support APIs of different languages officially like Go, Java, Python, Node.js, PHP, Ruby, etc. It also supports a few third party libraries for, among others, Haskell, Lisp, .NET, Perl, Rust and Scala. A complete listing is available at

InfluxDb Documentation ==> API Client Libraries.

Input protocols and service plugins

By default, InfluxDb supports HTTP transport and data formats as per the line protocol, with the syntax shown in Figure 1.

Measurement	,	Tag Set	space	Field Set	space	Time stamp

Figure 1: Line protocol syntax

Here, tagset and fieldset are a combination of one or more key-value pairs, separated by a comma. Timestamp is a value as per the RFC3339 format, with nano precision. Measurement name and at least one field key-value pair is a must, while tagset or timestamp are optional.

SQL analogy

In comparison with SQL, measurements are like tables and points are like records; fields are equivalent to unindexed columns with numeric data only, on which aggregations can be applied; whereas tags are like indexed columns with any type of data (typically strings). A combination of a measurement with a tagset is known as a series. Influx doesn't believe in pre-defined schema, i.e., in the following examples, measurements like temperature, humidity and pressure are created on-the-fly with suitable tag pairs.

Cross measurement joins are not allowed in InfluxQL; so, if you have any such plan, consider a single measurement with different tags. But, preferably, keep values with different meanings under separate measurements. Also, instead of encoding measurements or tag names with multiple pieces of information, use separate tags for each piece.

Let's assume a few points have been written into the city weather database, as follows:

```
temperature,city=delhi,metric=celsius  value=35
      1488731314272947906
humidity,city=pune,sensor=dht  internal=24,external=32
pressure,city=mumbai,sensor=bmp180  value=72
```

Data can be collected through other channels like UDP and other data formats, known as protocols supported by Graphite and OpenTSDB. This means InfluxDb can act like a drop-in replacement for OpenTSDB or Graphite, using the respective input plugins. It can also store the data sent by CollectD, such as performance and monitoring related metrics in a DevOps scenario.

Influx Query Language (QL)

InfluxQL supports SQL like query operations with typical